

Correction TP03**Matière : ATELIER DEVELOPPEMENT MOBILE AVANCE****Classes : SEM31**

Solution1 : Application Web

- 1- Lancer le navigateur (Browser) du Smartphone
 - 2- Consulter la page "http ://votre_adresse_ip :80/JQM_PHP_Bib/bib /index.html"
- Aucune ligne de code
Fonctionne sur tous les types de smartphone

Solution1 : Application Hybride

- 1- Créer une Application android
 - 2- Ajouter un WebView à l'activité
 - 3- Ajouter la permission INTERNET à androidManifest.xml
 - 4- Ecrire le code de la classe MainActivity
- Fonctionne seulement sur les terminaux android

```
package com.gestion.bib
class MainActivity : AppCompatActivity() {
    private lateinit var amb: ActivityMainBinding;
    private lateinit var webV1: WebView
    // Correction de l'espace accidentel dans l'URL
    private val url = "http://192.168.1.19/JQM_PHP_Bib/bib/index.html"
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        initGraphique()
    }
    private fun initGraphique() {
        amb= ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)
        webV1 = findViewById(R.id.webV1)
        // Empêche l'ouverture du navigateur externe lors d'un clic sur un lien
        webV1.webViewClient = WebViewClient()
        // Configuration des paramètres du WebView via la fonction 'apply' de Kotlin
        webV1.settings.apply {
            loadsImagesAutomatically = true
            javaScriptEnabled = true
            domStorageEnabled = true // Recommandé pour les applications web modernes
        }
        webV1.loadUrl(url)
    }
}
```

Solution3 : Application Native

```
package com.gestion.bib
```

```
class Config {
```

```
    companion object {
```

```
        const val JSON_BIB_URL = "http://192.168.1.19/JSON_PHP_Bib/bib"
```

```
    }
```

```
}
```

```
package com.gestion.bib
```

```
data class Livre(
```

```
    val id: Int,
```

```
    val titre: String,
```

```
    val nbpage: Int
```

```
) {
```

```
    override fun toString(): String {
```

```
        return "$id-$titre-$nbpage"
```

```
    }
```

```
}
```

```
package com.gestion.bib
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var amb: ActivityMainBinding
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        initGraphique()
```

```
    }
```

```
    private fun initGraphique() {
```

```
        amb = ActivityMainBinding.inflate(layoutInflater)
```

```
        setContentView(amb.root)
```

```
        ajouterEcouleurs()
```

```
    }
```

```
    private fun ajouterEcouleurs() {
```

```
        amb.btnAjout.setOnClickListener { ajouter() }
```

```
        amb.btnConsultation.setOnClickListener { consulter() }
```

```
        amb.btnModification.setOnClickListener { modifier() }
```

```
        amb.btnSuppression.setOnClickListener { supprimer() }
```

```
    }
```

```
    fun ajouter() {
```

```
        val i = Intent(this, AjoutActivity::class.java)
```

```
        startActivity(i)
```

```
    }
```

```
    fun consulter() {
```

```
        val i = Intent(this, ConsultationActivity::class.java)
```

```
        startActivity(i)
```

```
    }
```

```
    fun modifier() {
```

```
        val i = Intent(this, ModificationActivity::class.java)
```

```
        startActivity(i)
```

```
    }
```

```

fun supprimer() {
    val i = Intent(this, SuppressionActivity::class.java)
    startActivity(i)
}
}

package com.gestion.bib
class ConsultationActivity : AppCompatActivity() {
    private lateinit var amb: ActivityConsultationBinding
    private lateinit var adpLivre: ArrayAdapter<Livre>
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initGraphique()
    }
    private fun initGraphique() {
        amb = ActivityConsultationBinding.inflate(layoutInflater)
        setContentView(amb.root)

        // Initialisation de l'adaptateur pour la ListView
        adpLivre = ArrayAdapter(this, R.layout.simple_list_item_1)
        amb.lstLivre.adapter = adpLivre

        ajouterEcouleur()
        remplir()
    }
    private fun ajouterEcouleur() {
        amb.btnRetour.setOnClickListener {
            retourner()
        }
    }

    private fun retourner() {
        finish()
    }
    private fun remplir() {
        val queue: RequestQueue = Volley.newRequestQueue(this)
        val url = "${Config.JSON_BIB_URL}/liste/ListeLivres.php"

        val stringRequest = StringRequest(
            Request.Method.POST, url,
            Response.Listener { response ->
                try {
                    val r = JSONObject(response)
                    val a: JSONArray = r.getJSONArray("liste")
                    adpLivre.clear()
                    for (i in 0 until a.length()) {
                        val o: JSONObject = a.getJSONObject(i)
                        val id = o.getInt("id")
                        val titre = o.getString("titre")
                        val nbPage = o.getInt("nbPage")
                        val l = Livre(id, titre, nbPage)
                        adpLivre.add(l)
                    }
                } catch (error: JSONException) {
                    Toast.makeText(
                        this,
                        "Problème d'analyse JSON: ${error.message}",

```

```

        Toast.LENGTH_LONG
    ).show()
    },
    Response.ErrorListener { e ->
        Toast.makeText(this, "Problème d'appel HTTP: ${e.message}", Toast.LENGTH_LONG)
            .show()
    }
)
queue.add(stringRequest)
}
}

```

package com.gestion.bib

class AjoutActivity : AppCompatActivity() {

private lateinit var amb: ActivityAjoutBinding

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    initGraphique()
}

```

```

private fun initGraphique() {
    amb = ActivityAjoutBinding.inflate(layoutInflater)
    setContentView(amb.root)
    ajouterEcouteurs()
}

```

```

private fun ajouterEcouteurs() {
    amb.btnAjouter.setOnClickListener {
        ajouterLivre()
    }
}

```

```

    amb.btnRetour.setOnClickListener {
        retourner()
    }
}

```

```

private fun retourner() {
    finish()
}

```

```

private fun ajouterLivre() {
    val queue: RequestQueue = Volley.newRequestQueue(this)

```

```

// Utilisation de votre URL (supposant que Config.URL est défini ou votre constante)
val url = "${Config.JSON_BIB_URL}/ajout/Ajout.php"

```

```

val stringRequest = object : StringRequest(
    Request.Method.POST, url,
    Response.Listener { response ->
        try {
            val r = JSONObject(response)
            val etat = r.getString("ETAT")

```

```

        if (etat == "SUCCES") {
            amb.edTitre1.setText("")
            amb.edNbPage1.setText("")
            amb.edTitre1.requestFocus()
            Toast.makeText(this, "Livre ajouté avec succès", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Echec de l'ajout", Toast.LENGTH_LONG).show()
        }
    } catch (error: JSONException) {
        Toast.makeText(this, "Problème d'analyse JSON: ${error.message}",
Toast.LENGTH_LONG).show()
    }
},
Response.ErrorListener { e ->
    Toast.makeText(this, "Problème d'appel HTTP: ${e.message}",
Toast.LENGTH_LONG).show()
}
) {
    override fun getParams(): Map<String, String> {
        val headers = HashMap<String, String>()
        headers["titre"] = amb.edTitre1.text.toString()
        headers["nbPage"] = amb.edNbPage1.text.toString()
        return headers
    }
}

queue.add(stringRequest)
}
}

```

package com.gestion.bib

class SuppressionActivity : AppCompatActivity() {

private lateinit var amb: ActivitySuppressionBinding

private lateinit var adpLivre: ArrayAdapter<Livre>

override fun onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

initGraphique()

}

private fun initGraphique() {

amb = ActivitySuppressionBinding.inflate(layoutInflater)

setContentView(amb.root)

// Initialisation de l'adaptateur pour le Spinner

adpLivre = ArrayAdapter(this, R.layout.simple_list_item_1)

adpLivre.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)

amb.spLivre1.adapter = adpLivre

remplir()

ajouterEcouteur()

}

private fun ajouterEcouteur() {

amb.btnSupprimer.setOnClickListener {

supprimerLivre()

}

```

        amb.btnRetour.setOnClickListener {
            retourner()
        }
    }

    private fun remplir() {
        val queue: RequestQueue = Volley.newRequestQueue(this)
        val url = "${Config.JSON_BIB_URL}/liste/ListeLivres.php"

        val jsObjRequest = JsonObjectRequest(
            Request.Method.POST, url, null,
            Response.Listener { response ->
                try {
                    val a: JSONArray = response.getJSONArray("liste")
                    adpLivre.clear()
                    for (i in 0 until a.length()) {
                        val o: JSONObject = a.getJSONObject(i)
                        val id = o.getInt("id")
                        val titre = o.getString("titre")
                        val nbPage = o.getInt("nbPage")
                        val l = Livre(id, titre, nbPage)
                        adpLivre.add(l)
                    }
                } catch (e: JSONException) {
                    Toast.makeText(this, "Problème d'analyse JSON: ${e.message}",
                        Toast.LENGTH_LONG)
                        .show()
                }
            },
            Response.ErrorListener { error ->
                Toast.makeText(this, "Problème d'appel HTTP: ${error.message}",
                    Toast.LENGTH_LONG)
                    .show()
            }
        )
        queue.add(jsObjRequest)
    }

    private fun supprimerLivre() {
        if (amb.spLivre1.selectedItemPosition >= 0) {
            val l = amb.spLivre1.selectedItem as Livre
            val queue: RequestQueue = Volley.newRequestQueue(this)
            val url = "${Config.JSON_BIB_URL}/suppression/Suppression.php"

            val stringRequest = object : StringRequest(
                Method.POST, url,
                Response.Listener { response ->
                    try {
                        val r = JSONObject(response)
                        val etat = r.getString("ETAT")
                        if (etat == "SUCCES") {
                            Toast.makeText(this, "Livre supprimé avec succès", Toast.LENGTH_SHORT)
                                .show()
                            remplir() // Actualiser la liste du Spinner
                        } else {
                            Toast.makeText(this, "Echec de la suppression", Toast.LENGTH_LONG)

```

```

        .show()
    }
} catch (error: JSONException) {
    Toast.makeText(
        this,
        "Problème d'analyse JSON: ${error.message}",
        Toast.LENGTH_LONG
    ).show()
}
},
Response.ErrorListener { e ->
    Toast.makeText(this, "Problème d'appel HTTP: ${e.message}",
        Toast.LENGTH_LONG)
        .show()
    }
} {
    override fun getParams(): Map<String, String> {
        val headers = HashMap<String, String>()
        headers["id"] = l.id.toString()
        return headers
    }
}
queue.add(stringRequest)
} else {
    Toast.makeText(this, "Veuillez sélectionner un livre à supprimer",
        Toast.LENGTH_SHORT)
        .show()
}
}
}

private fun retourner() {
    finish()
}
}
}

```

package com.gestion.bib

class ModificationActivity : AppCompatActivity() {

private lateinit var amb: ActivityModificationBinding

private lateinit var adpLivre: ArrayAdapter<Livre>

override fun onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

initGraphique()

}

private fun initGraphique() {

amb = ActivityModificationBinding.inflate(layoutInflater)

setContentView(amb.root)

// Initialisation de l'adaptateur pour le Spinner

adpLivre = ArrayAdapter(this, R.layout.simple_list_item_1)

adpLivre.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)

amb.spLivre2.adapter = adpLivre

remplir()

ajouterEcouteur()

}

```

private fun remplir() {
    val queue: RequestQueue = Volley.newRequestQueue(this)
    val url = "${Config.JSON_BIB_URL}/liste/ListeLivres.php"
    val jsObjRequest = JsonObjectRequest(
        Request.Method.POST, url, null,
        Response.Listener { response ->
            try {
                val a: JSONArray = response.getJSONArray("liste")
                adpLivre.clear()
                for (i in 0 until a.length()) {
                    val o: JSONObject = a.getJSONObject(i)
                    val id = o.getInt("id")
                    val titre = o.getString("titre")
                    val nbPage = o.getInt("nbPage")
                    val l = Livre(id, titre, nbPage)
                    adpLivre.add(l)
                }
            } catch (e: JSONException) {
                Toast.makeText(
                    this,
                    "Problème d'analyse JSON: ${e.message}",
                    Toast.LENGTH_LONG
                ).show()
            }
        },
        Response.ErrorListener { error ->
            Toast.makeText(
                this,
                "Problème d'appel HTTP: ${error.message}",
                Toast.LENGTH_LONG
            ).show()
        }
    )
    queue.add(jsObjRequest)
}

private fun ajouterEcouleur() {
    amb.btnModifier.setOnClickListener {
        modifierLivre()
    }
    amb.btnRetour.setOnClickListener {
        retourner()
    }
    // Écouteur de sélection dans le Spinner pour pré-remplir les champs
    amb.spLivre2.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
        override fun onItemSelected(
            parent: AdapterView<*>,
            view: View?,
            position: Int,
            id: Long
        ) {
            actualiser()
        }
        override fun onNothingSelected(parent: AdapterView<*>) {
            // Rien à faire
        }
    }
}

```

```

private fun actualiser() {
    val l = amb.spLivre2.selectedItem as? Livre
    if (l != null) {
        amb.edTitre01.setText(l.titre)
        amb.edNbPage01.setText(l.nbpage.toString())
    }
}
private fun retourner() {
    finish()
}
private fun modifierLivre() {
    if (amb.spLivre2.selectedItemPosition >= 0) {
        val l = amb.spLivre2.selectedItem as Livre
        val queue: RequestQueue = Volley.newRequestQueue(this)
        val url = "${Config.JSON_BIB_URL}/modification/Modification.php"
        val stringRequest = object : StringRequest(
            Request.Method.POST, url,
            Response.Listener { response ->
                try {
                    val r = JSONObject(response)
                    val etat = r.getString("ETAT")
                    if (etat == "SUCCES") {
                        Toast.makeText(this, "Livre modifié avec succès", Toast.LENGTH_SHORT)
                            .show()
                        remplir() // Actualiser la liste du Spinner
                    } else {
                        Toast.makeText(this, "Echec de la modification", Toast.LENGTH_LONG)
                            .show()
                    }
                } catch (error: JSONException) {
                    Toast.makeText(
                        this,
                        "Problème d'analyse JSON: ${error.message}",
                        Toast.LENGTH_LONG
                    ).show()
                }
            },
            Response.ErrorListener { e ->
                Toast.makeText(this, "Problème d'appel HTTP: ${e.message}",
                    Toast.LENGTH_LONG).show()
            }
        ) {
            override fun getParams(): Map<String, String> {
                val headers = HashMap<String, String>()
                headers["id"] = l.id.toString()
                headers["titre"] = amb.edTitre01.text.toString()
                headers["nbPage"] = amb.edNbPage01.text.toString()
                return headers
            }
        }
        queue.add(stringRequest)
    } else {
        Toast.makeText(this, "Veuillez sélectionner un livre à modifier", Toast.LENGTH_SHORT)
            .show()
    }
}
}
}

```